Universidad de Granada

Periféricos y Dispositivos de Interfaz Humana
Curso 2025/2026

# Práctica 2. Uso de bibliotecas de programación de interfaces de usuario en modo texto

*Autora:*
Inés Ruiz Sánchez

3 de abril de 2026

UNIVERSIDAD
DE GRANADA

# Índice

**1. Introducción** **2**

**2. Requisitos mínimos** **2**
2.1. Instalación . . . 2
2.2. Ej1 - Hola mundo . . . 3
2.3. Ej2 - Ventana . . . 4
2.4. Ej3 - Pelota . . . 7
2.5. Juego sencillo . . . 8

**3. Requisitos ampliados** **11**
3.1. Pantalla de inicio . . . 11
3.2. Pantalla resumen . . . 13

**4. Anexo** **14**
4.1. Ejemplo completo para *ej1* . . . 14
4.2. Ejemplo completo para *ej2* . . . 14
4.3. Ejemplo modificado de *ej2* . . . 15
4.4. Ejemplo completo para *ej3* . . . 16
4.5. Código completo para el juego Pong . . . 16
4.6. Código completo para Pong con pantallas . . . 19

# Índice de figuras

1. Comandos de instalación . . . 2
2. Continuar instalación . . . 3
3. Ejecución de hello.c . . . 4
4. Fichero ventana.c y su compilación . . . 5
5. Ejecución de ventana.c . . . 6
6. Ejecución de ventana.c con modificación . . . 7
7. Ejecución de pelotita.c . . . 8
8. Ejecución de pong.c . . . 11
9. Pantalla de inicio . . . 12
10. Pantalla de fin de juego . . . 14

# 1. Introducción

"ncurses" es una biblioteca de programación que provee una API que permite al programador escribir interfaces basadas en texto.

Los objetivos concretos de esta práctica son:

- instalar la librería *ncurses* en Linux
- crear programas sencillos basados en *ncurses*

Nota: Las tildes en los trozos de código no se reconocen en LATEX por lo que el carácter no se escribe. Es por eso por lo que o bien no aparece un caracter o bien el caracter no tiene tilde, pareciendo una falta ortográfica.

# 2. Requisitos mínimos

Como requisitos mínimos se deben realizar y documentar adecuadamente las siguientes tareas:

- Instalar la librería ncurses, crear los programas de ejemplo ofrecidos (ej1, ej2 y ej3), y comprobar su funcionamiento.
- Crear un juego sencillo tipo "pong" partiendo del ejemplo del movimiento de la pelotita.

## 2.1. Instalación

Esta práctica se realiza en una máquina virtual con Ubuntu 22.
Instalar la librería necesaria:

```
sudo apt-get update
sudo apt-get install libncurses5-dev libncursesw5-dev
```

Listing 1: Comandos para instalar la librería

A continuación se muestra una imagen usando el segundo comando[1]:

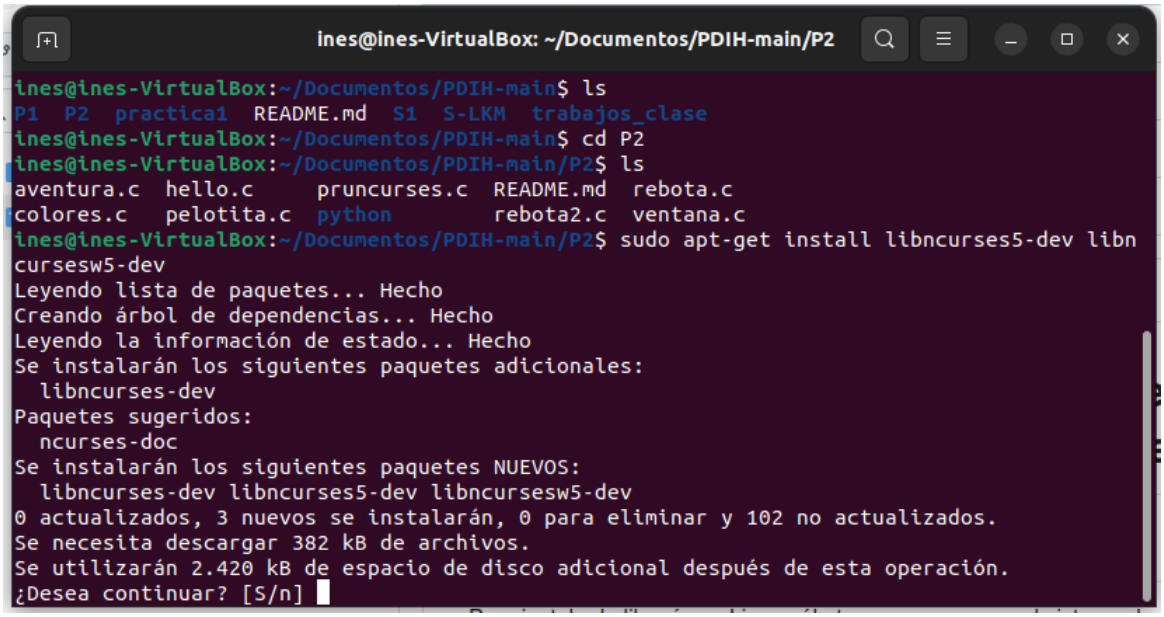


Figura 1: Comandos de instalación

Pulsamos la tecla S para continuar.

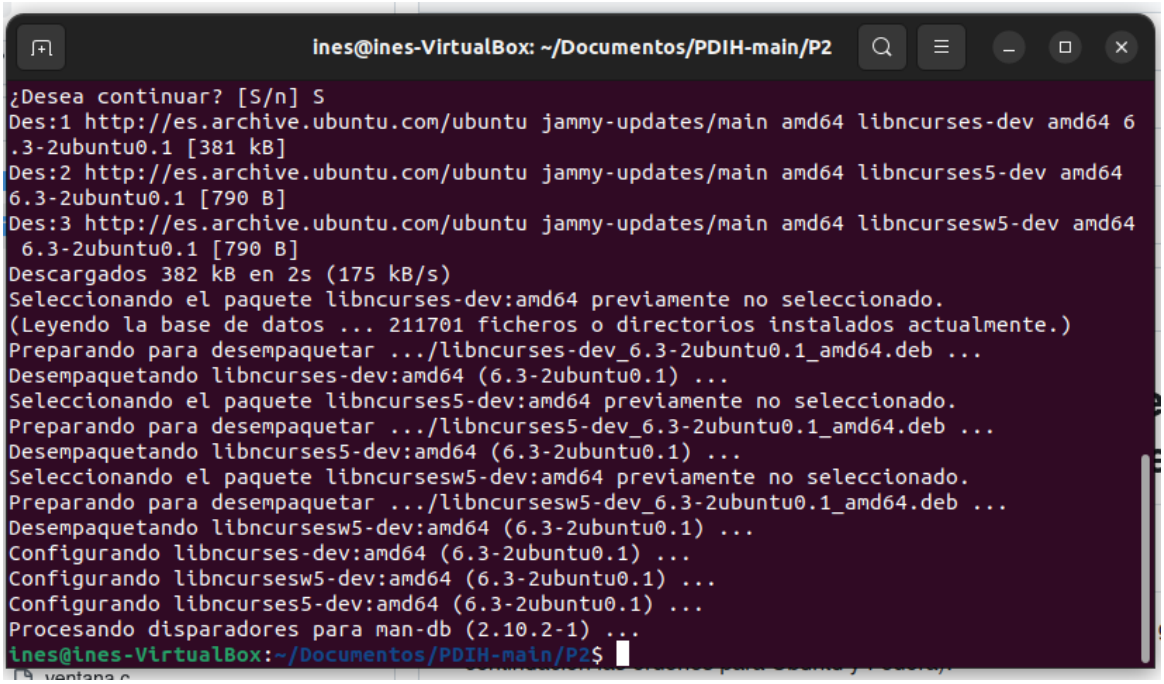


Figura 2: Continuar instalación

La instalación ha terminado con exito.

## 2.2. Ej1 - Hola mundo

Ejemplo proporcionado por el profesor:

```c
#include <ncurses.h>
#include <stdio.h>
int main() {
    initscr(); // Reserva memoria para la ventana principal
    printw("Hello␣World!"); // Lo escribe en la memoria
    refresh(); // Pasa lo que hay en memoria a la pantalla para que se vea
    getch();
    endwin(); // Cierra el modo ncurses
    return 0;
}
```

Listing 2: Fichero hello.c

Orden para compilar:

```
gcc hello.c -o hello -lncurses
```

Listing 3: Orden para compilar ej1

Al ejecutar el programa[2] se verá:

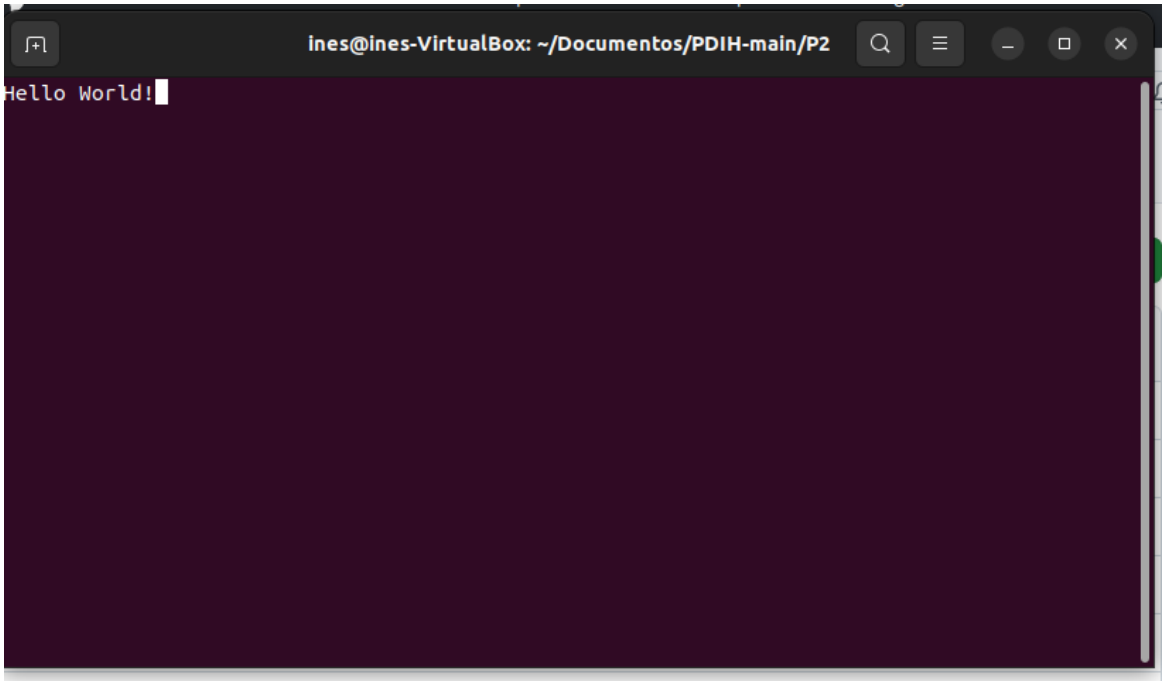


Figura 3: Ejecución de hello.c

El código de este ejercicio se encuentra en el anexo 4.1.

## 2.3. Ej2 - Ventana

Se trata del fichero *ventana.c*, incluido en el anexo 4.2

```c
int main(void) {
    int rows, cols;

    initscr();

    if (has_colors() == FALSE) { // Comprobar soporte de color
        endwin();
        printf("Your terminal does not support color\n");
        exit(1);
    }

    start_color(); // Activar soporte color

    // Crear combinaci n de colores
    init_pair(1, COLOR_YELLOW, COLOR_GREEN); // Texto amarillo, fondo verde
    init_pair(2, COLOR_BLACK, COLOR_WHITE);
    init_pair(3,COLOR_WHITE,COLOR_BLUE);
    clear();

    refresh();
    getmaxyx(stdscr, rows, cols);

    WINDOW *window = newwin(rows,cols,0,0); // Crea una ventana con las dimensiones de la terminal
    wbkgd(window, COLOR_PAIR(3));
    box(window, '|', '-'); // Dibuja el marco de la ventana (WINDOW, caracter vertical, caracter horizontal)

    mvwprintw(window, 10, 10, "una cadena");
    wrefresh(window);

    getch();
```

```
    endwin();
    return 0;
}
```

Listing 4: Programa ventana.c

Compilar con la orden:

```
    gcc ventana.c -o ventana -lncurses
```

Listing 5: Compilación de ventana.c

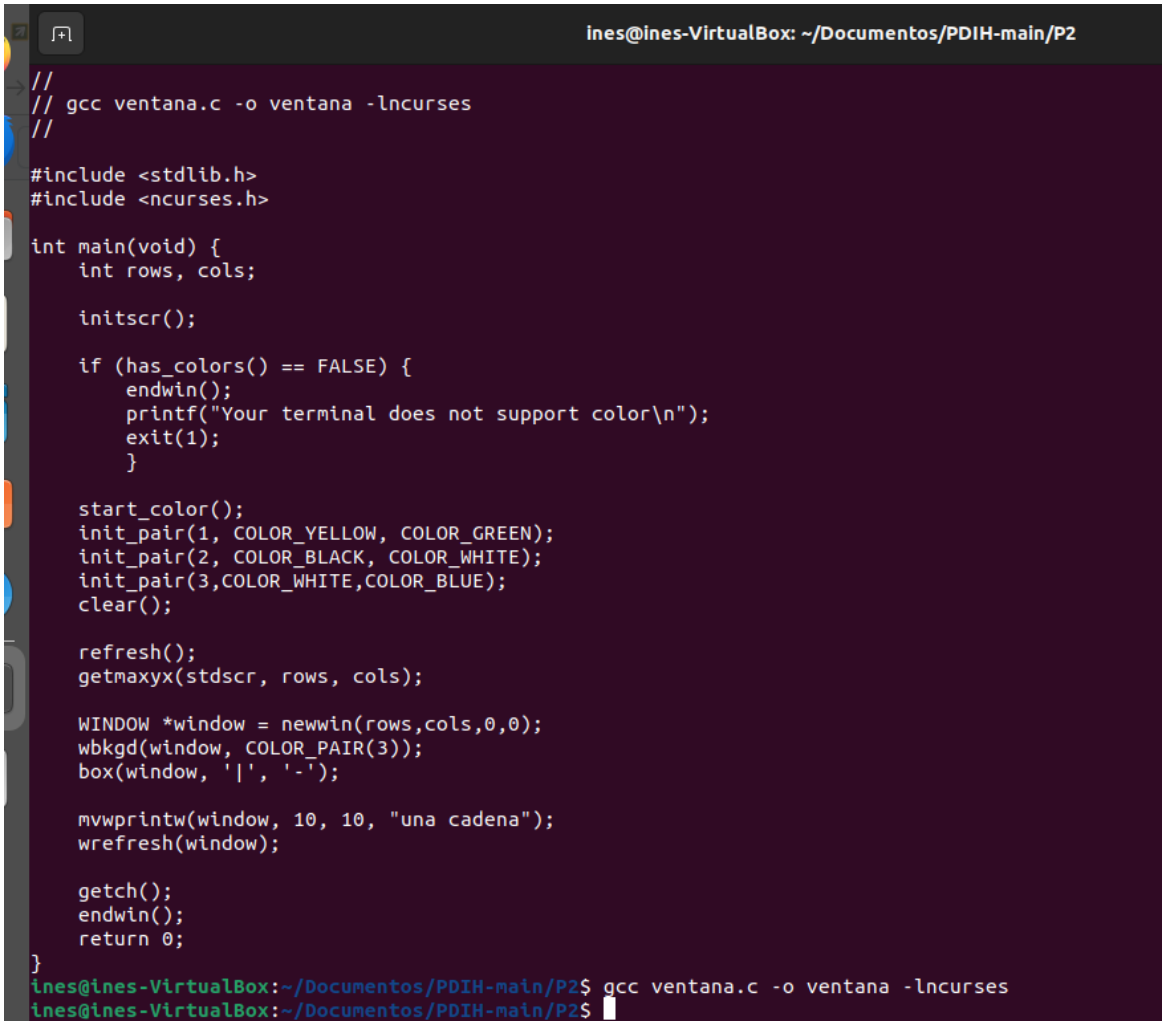


Figura 4: Fichero ventana.c y su compilación

Ejecución del programa *ventana*:

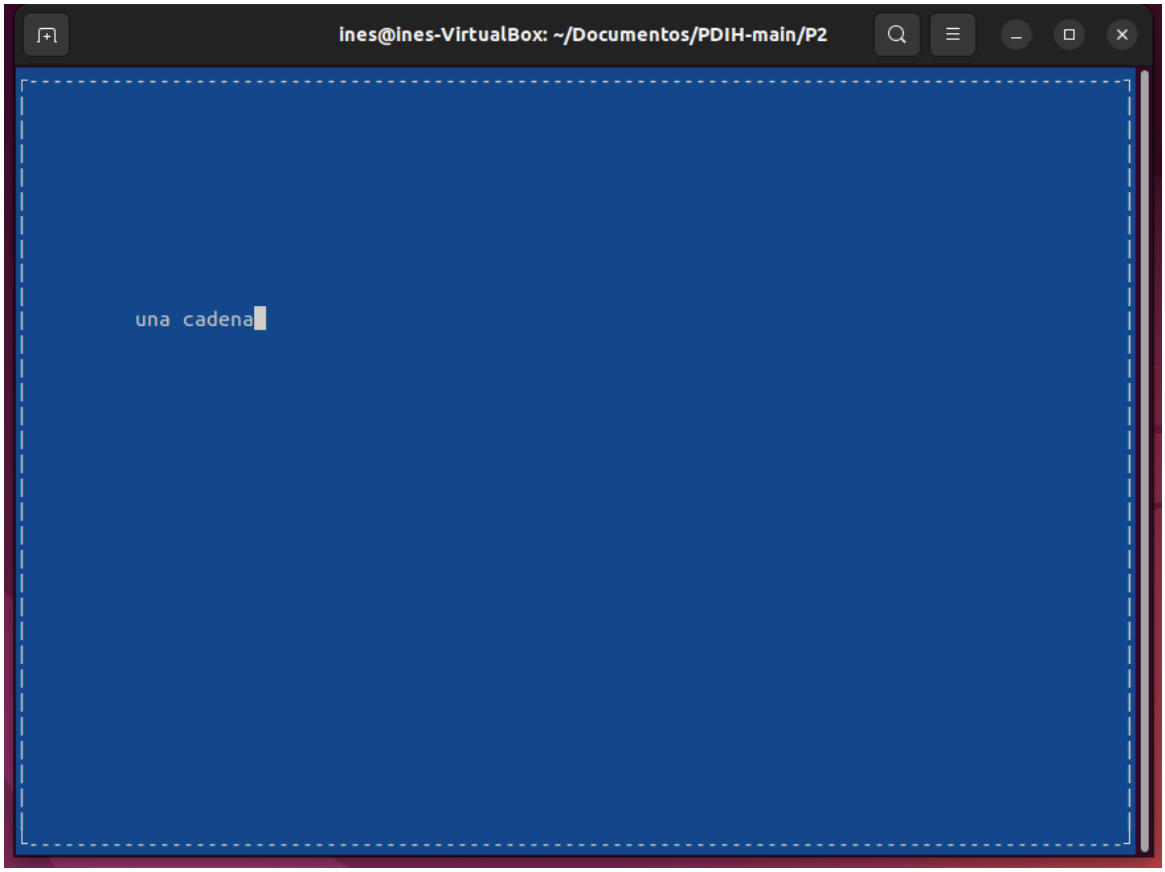


Figura 5: Ejecución de ventana.c

Una modificación de ese ejemplo se puede ver en el anexo 4.2. Las líneas que se han cambiado con respecto al fichero inicial son:

```
//getmaxyx(stdscr, rows, cols);
rows=10;
cols=40;

WINDOW *window = newwin(rows,cols,5,10); // Modificaci n del tama o
wbkgd(window, COLOR_PAIR(1)); // Cambio de colores: amarillo y verde
box(window, '*', '*'); // Modificaci n de marco
```

Listing 6: Modificación de ventana.c

Ejecución:

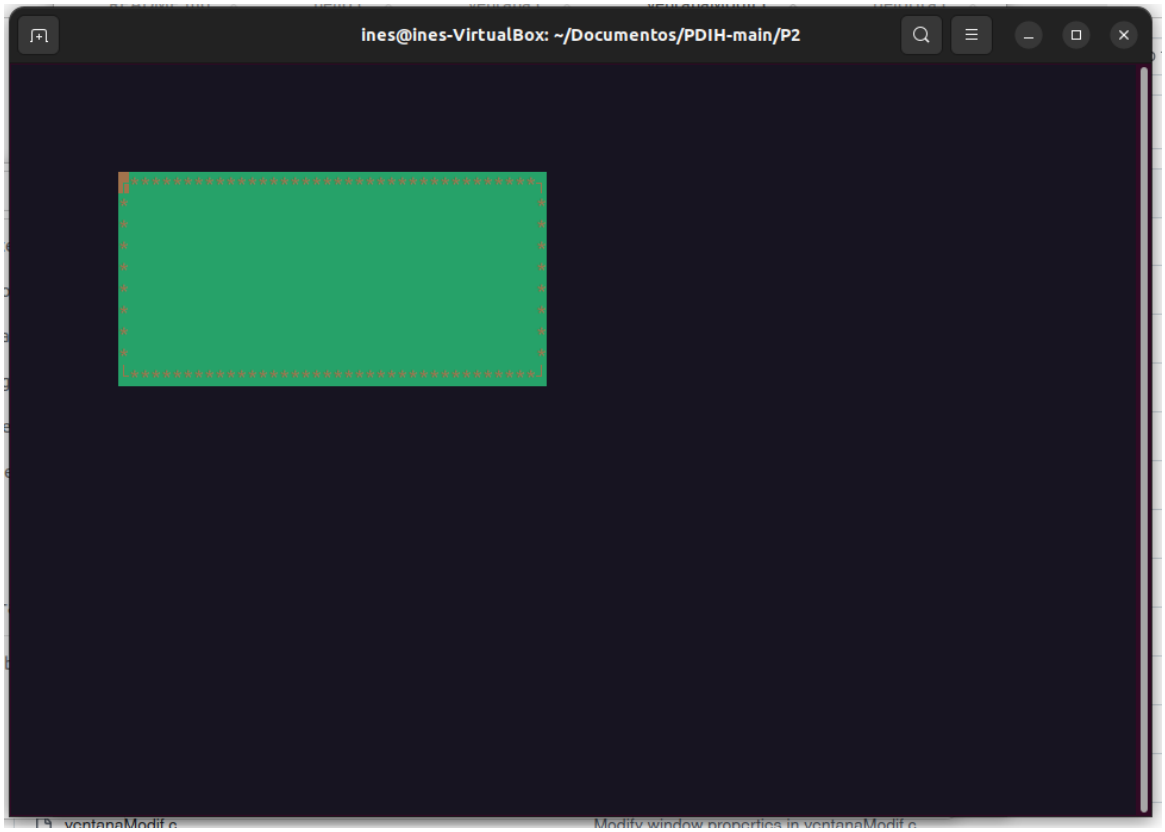


Figura 6: Ejecución de ventana.c con modificación

Se pueden observar los cambios:

- con la función *box* se cambian los caracteres para el marco: *
- con la función *wbkgd* se cambia el color de la ventana: verde para el fondo y amarillo para los caracteres
- con la función *newwin* se modifica el tamaño de la ventana indicando alto, ancho, inicio de y, inicio de x

## 2.4. Ej3 - Pelota

Fichero *pelotita.c* proporcionado por el profesor 4.4:

```c
#include <ncurses.h>
#include <unistd.h>

#define DELAY 30000

int main(int argc, char *argv[]) {
 int x = 0, y = 0;
 int max_y = 20, max_x = 20;
 int next_x = 0;
 int direction = 1;

 initscr();
 noecho(); // Para no mostrar por pantalla las teclas que se pulsan
 curs_set(FALSE); // Ocultar el cursor

 while(1) {
    clear();
    mvprintw(y, x, "o");
    refresh();

    usleep(DELAY); // Pausa para que nos de tiempo a ver la pelota
```

```c

    next_x = x + direction;

    if (next_x >= max_x || next_x < 0) {
        direction*= -1;
    } else {
        x+= direction;
    }
 }

 endwin();
}
```

Listing 7: Programa pelotita.c

Para compilar hay que usar la siguiente orden:

```c
    gcc pelotita.c -o pelotita -lncurses
```

Listing 8: Compilacion pelotita.c

Ejecución:

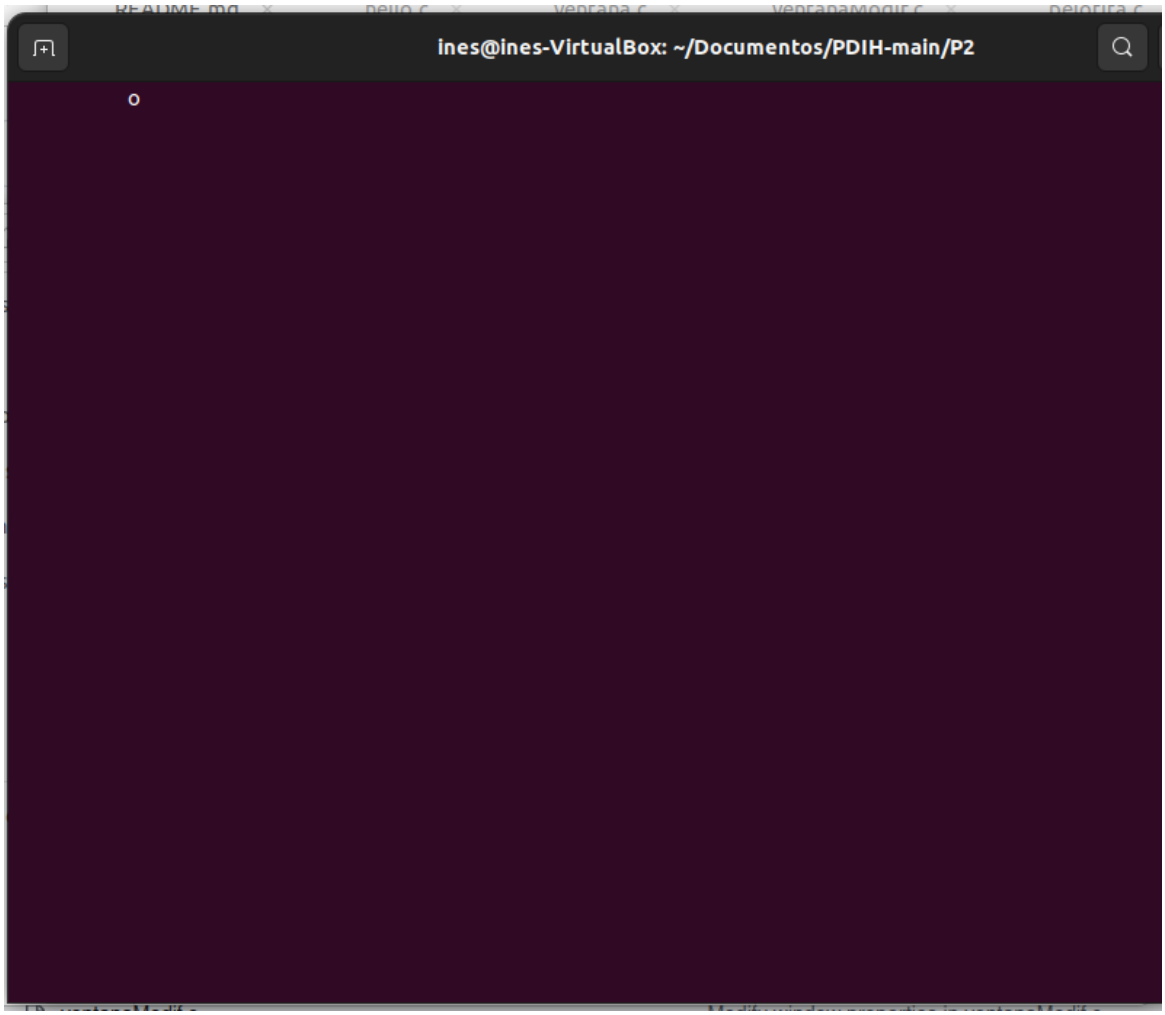


Figura 7: Ejecución de pelotita.c

## 2.5. Juego sencillo

Para crear el juego partimos del fichero pelotita.c (anexo 4.4), pero hay que añadirle las palas y la lógica de la puntuación. Los cambios han sido:

- Aumentar el valor de la constante *DELAY* para que la pausa sea mayor, lo que implica que la velocidad de la pelota es menor. Disminuir el valor de esta constante implica hacer el juego más difícil.
- Añadir movimiento en el Eje Y para la pelota.

```
        int pelota_x = max_x /2;
        int pelota_y = max_y /2;
        int dir_x = 1;
        int dir_y = 1;
        ...
        pelota_x += dir_x ;
        pelota_y += dir_y ;
```

Listing 9: Movimiento de pelota

- Añadir las dos palas, solo con movimiento en Eje Y.

```
        int pala_izq_y = ( max_y /2) - ( LONGITUD_PALA /2) ;
        int pala_der_y = ( max_y /2) - ( LONGITUD_PALA /2) ;
```

Listing 10: Movimiento de palas

- Capturar las teclas de juego con la función *getch()*

```
        int tecla = getch();
        if (tecla == 'w' || tecla == 'W')
            if (pala_izq_y > 0) pala_izq_y--;
        if (tecla == 's' || tecla == 'S')
            if (pala_izq_y < max_y - LONGITUD_PALA) pala_izq_y++;
        if (tecla == KEY_UP)
            if (pala_der_y > 0) pala_der_y--;
        if (tecla == KEY_DOWN)
            if (pala_der_y < max_y - LONGITUD_PALA) pala_der_y++;
```

Listing 11: Captación de teclas

Para poder capturar las teclas especiales (flechas) he usado la función *keypad()*.

- El jugador 1 usa las teclas $W$ y $S$ para mover su pala.
- El jugador 2 usa las teclas con **flechas** para mover su pala.

- Marcador de puntos

```
        int puntos_j1 = 0;
        int puntos_j2 = 0;
        ...
        if ( pelota_x <= 0) { // Punto para el jugador 2
            puntos_j2 ++;
            pelota_x = max_x / 2; pelota_y = max_y / 2;
            dir_x = 1;
        } else if ( pelota_x >= max_x - 1) { // Punto para el jugador 1
            puntos_j1 ++;
            pelota_x = max_x / 2; pelota_y = max_y / 2;
            dir_x = -1;
        }
```

Listing 12: Puntuación

Para este juego uso un tablero fijo de $30x60$ para asegurar que el área de juego se dibuje correctamente puesto que inicialmente intenté usar la función *getmaxyx()* pero daba problemas. Además, se usa la función *nodelay()* para que la función *getch()* no bloquee el movimiento de la pelota.

La lógica de las **colisiones** es la siguiente:

- **Colisión con el techo y el suelo**. En este caso se comprueban las coordenadas verticales para que la pelota no se salga del marco superior o inferior del juego. El rebote se produce cuando la pelota alcanza la fila *1* (justo debajo del techo) o la fila *max_y -2* (justo encima del suelo). Se multiplica por $-1$ para cambiar la dirección de *dir_y*.

- **Colisión con las palas**. Para ello se debe cumplir que la pelota haya llegado a la columna inmediatamente anterior a la pala y que su altura coincida con la posición actual de la pala.

  - Para la pala izquierda hay que comprobar que la pelota está en la columna 3 ya que la pala se dibuja en la columna 2. Y hay que comprobar si su coordenada $Y$ se encuentra dentro del rango de la pala (entre *pala_izq_y* y *pala_izq_y + LONGITUD_PALA*).
  - Para la pala derecha hay que comprobar que la pelota ha llegado a la columna *max_x = 4* porque la pala está fija en la columna *max_x = 3*.

  Si la pelota ha colisionado con alguna de las palas hay que invertir su dirección horizontal: *dir_x *= -1*.

- **Fuera de la pista (límites)**. Si la pelota no colisiona con ninguna pala llegará hasta los extremos laterales de la pantalla (*pelota_x* $<= 0$ o *pelota_x* $>=$ *max_x* $= 1$). En este caso se anota un punto al jugador contrario y se coloca la pelota en el centro del tablero.

```
// Rebote en techo y suelo
if(pelota_y <=1 || pelota_y >= max_y -2){
    dir_y *= -1;
}

// Rebote en la pala izquierda
if (pelota_x == 3 && pelota_y >= pala_izq_y && pelota_y < pala_izq_y +
    LONGITUD_PALA) {
    dir_x *= -1;
}

// Rebote en la pala derecha
else if (pelota_x == max_x - 4 && pelota_y >= pala_der_y && pelota_y < pala_der_y
     + LONGITUD_PALA) {
    dir_x *= -1;
}

// Puntuacion
if (pelota_x <= 0) { // Punto para el jugador 2
        puntos_j2++;
        pelota_x = max_x / 2; pelota_y = max_y / 2;
        dir_x = 1;
} else if (pelota_x >= max_x - 1) { // Punto para el jugador 1
        puntos_j1++;
        pelota_x = max_x / 2; pelota_y = max_y / 2;
        dir_x = -1;
}
```

Listing 13: Colisiones

Para compilar hay que usar la siguiente orden[14]:

```
gcc pong.c -o pong -lncurses
```

Listing 14: Compilación pong.c

Ejemplo de ejecución: [8]

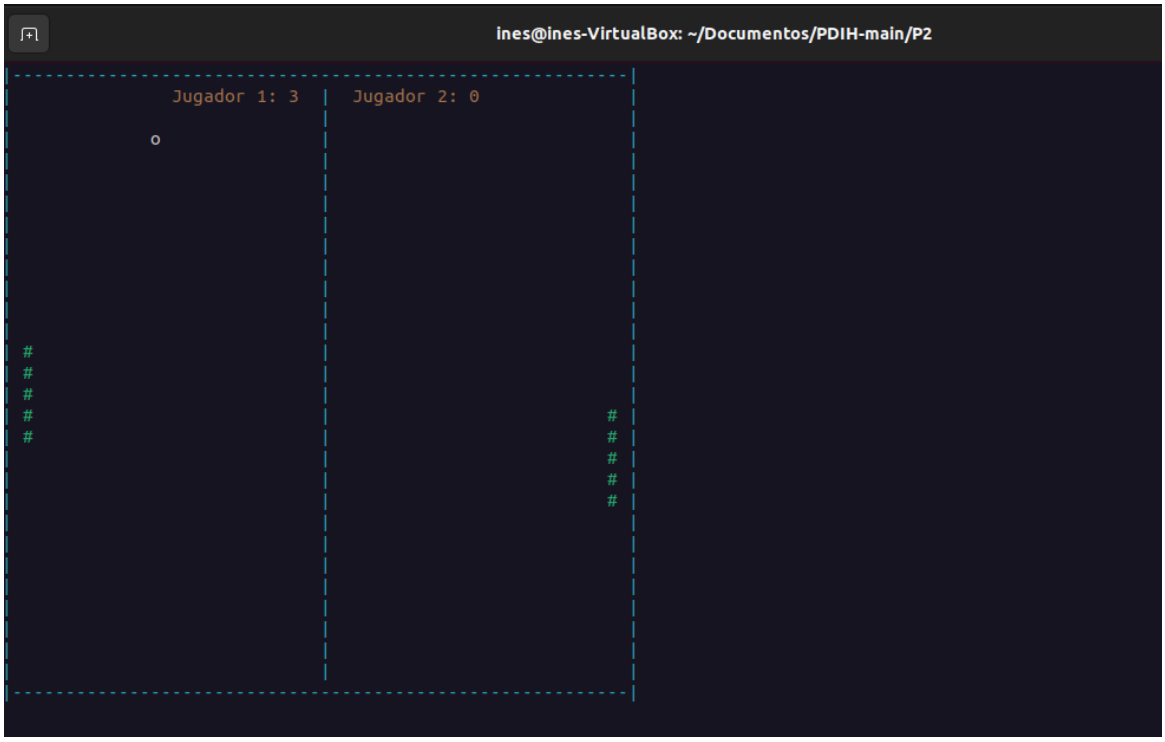


Figura 8: Ejecución de pong.c

# 3. Requisitos ampliados

Como requisitos ampliados (opcionales para subir nota) se propone:

- que al iniciar el juego se muestre una **pantalla de bienvenida** en la que se muestren los datos de quienes han realizado el juego y explicando los controles de juego (p.ej. un recuadro con la explicación). Tras una pausa o pulsación de tecla se iniciará el juego en sí mismo.
- que al terminar cada partida se muestre una **pantalla de resumen** mostrando el marcador final y felicitando al ganador. Se dará la opción de volver a jugar o terminar el programa.

Para tener un código más limpio y estructurado he separado la lógica del juego del diseño de las pantallas. Es por eso que la creación de cada pantalla está en una función, que será llamada en el *main*.

Parto del código de fichero pong.c (anexo 4.4) y realizo los siguientes cambios:

- Disminuir la velocidad de la pelota para que sea más asequible el juego. Esto se hace modificando la constante $DELAY$ 50000 al principio del fichero.
- Establecer un máximo de puntos para ganar con una constante: $PUNTOS_PARA_GANAR = 5$.

## 3.1. Pantalla de inicio

En el juego del *pong*, al iniciar el juego se mostrará una pantalla de bienvenida en la que se muestren los datos de quienes han realizado el juego y explicando los controles de juego (p.ej. un recuadro con la explicación). Tras una pausa o pulsación de tecla se iniciará el juego en sí mismo.

La función desactiva el modo continuo con $nodelay(stdscr, FALSE)$ para esperar a la pulsación de una tecla con *getch()*. Al pulsar cualquier tecla la función termina y devuelve el control al *main*, devolviendo *nodelay* a $TRUE$ para iniciar el movimiento de la pelota.

```
void pantalla_bienvenida(int max_y, int max_x){
    nodelay(stdscr, FALSE); // Para que getch() espere la pulsacion de una tecla
    clear();

```

```
    // Dibujar marco de presentacion
    attron(COLOR_PAIR(4));
    for(int i = 0; i < max_x; i++) { // Techo y suelo
            mvprintw(0, i, "-"); mvprintw(max_y - 1, i, "-");
    }
    for(int i = 0; i < max_y; i++) { // Paredes
            mvprintw(i, 0, "|"); mvprintw(i, max_x - 1, "|");
    }
    attroff(COLOR_PAIR(4));

    attron(COLOR_PAIR(3));
    mvprintw(max_y/2 - 5, max_x/2 - 6, "JUEGO␣PONG");
    attroff(COLOR_PAIR(3));

    mvprintw(max_y/2 - 3, max_x/2 - 15, "Autora:␣Ines␣Ruiz␣Sanchez");
    mvprintw(max_y/2 - 1, max_x/2 - 20, "J1␣(Pala␣Izquierda):␣W␣(Arriba)␣/␣S␣(Abajo)"
        );
    mvprintw(max_y/2 + 1, max_x/2 - 20, "J2␣(Pala␣Derecha):␣Flecha␣Arriba␣/␣Abajo");
    mvprintw(max_y/2 + 4, max_x/2 - 18, "Pulsa␣cualquier␣tecla␣para␣empezar␣...");

    refresh();
    getch(); // Esperar tecla
}
```

Listing 15: Pantalla de bienvenida

Para ver el código completo: anexo4.6.

Ejemplo de pantalla de inicio: [9]

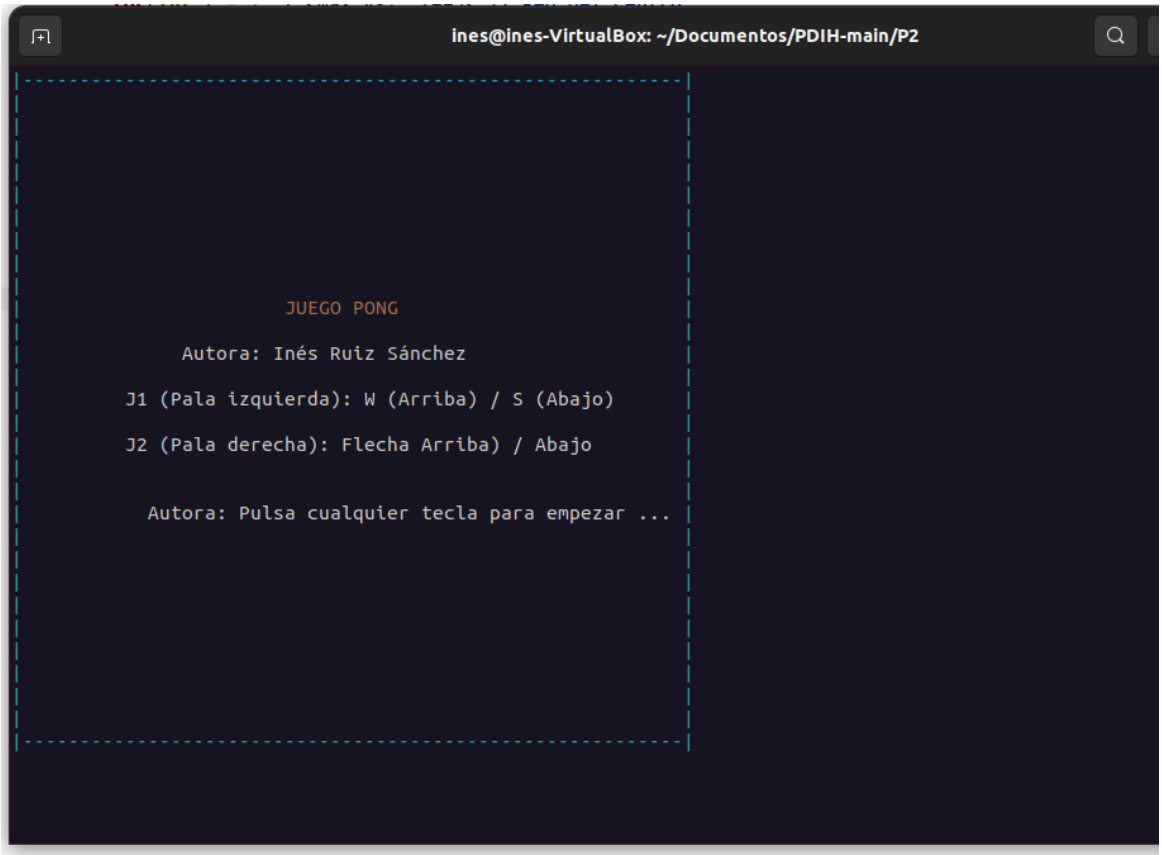


Figura 9: Pantalla de inicio

## 3.2. Pantalla resumen

En el juego del *pong*, al terminar cada par da se mostrará una pantalla de resumen mostrando el marcador final y felicitando al ganador. Se dará la opción de volver a jugar o terminar el programa.

La función recibe como argumentos las dimensiones de la pantalla (max_y, max_x), la puntuación de ambos jugadores y un entero identificando al ganador.

Además de mostrar la información, la función entra en un bucle infinito con *getch()* bloqueante para preguntar si el usuario quiere volver a jugar o parar (teclas 'S' o 'N'). Si el usuario pulsa 'S' la función devuelve 1, Si el usuario pulsa 'N' devuelve 0. Ese valor se tiene en cuenta en el bucle del *main while*(*jugar_de_nuevo*) para reiniciar partida o salir del programa.

```c
int pantalla_final(int max_y, int max_x, int ganador, int puntos_j1, int puntos_j2)
    nodelay(stdscr, FALSE); // sperar a que el usuario responda
    clear();

    // Marco de la pantalla final
    attron(COLOR_PAIR(4));
    for(int i = 0; i < max_x; i++) { mvprintw(0, i, "-"); mvprintw(max_y - 1, i, "-")
        ; } // Techo y suelo
    for(int i = 0; i < max_y; i++) { mvprintw(i, 0, "|"); mvprintw(i, max_x - 1, "|")
        ; } // Paredes
    attroff(COLOR_PAIR(4));

    attron(COLOR_PAIR(3));
    mvprintw(max_y/2 - 4, max_x/2 - 8, "Fin del juego");
    attroff(COLOR_PAIR(3));

    if (ganador == 1) {
        mvprintw(max_y/2 - 2, max_x/2 - 15, "Felicidades Jugador 1, has ganado!");
    } else if (ganador == 2) {
        mvprintw(max_y/2 - 2, max_x/2 - 15, "Felicidades Jugador 2, has ganado!");
    } else {
        mvprintw(max_y/2 - 2, max_x/2 - 10, "Partida cancelada");
    }

    mvprintw(max_y/2, max_x/2 - 15, "Marcador Final");
    mvprintw(max_y/2, max_x/2 - 16, "J1: %d puntos | J2: %d puntos", puntos_j1,
        puntos_j2);
    mvprintw(max_y/2 + 3, max_x/2 - 17, "Pulsa S para volver a jugar y N para
        terminar. ");
    refresh();

    // Captar respuesta
    int respuesta;
    while(1) {
        respuesta = getch();
        if (respuesta == 'n' || respuesta == 'N' || respuesta == 'q' || respuesta ==
            'Q') {
            return 0;
        } else if (respuesta == 's' || respuesta == 'S') {
            return 1;
        }
    }
}
```

Listing 16: Pantalla de finalización

Para ver el código completo: anexo4.6.

Ejemplo de pantalla de inicio: [10]

Figura 10: Pantalla de fin de juego

# 4. Anexo

## 4.1. Ejemplo completo para *ej1*

Código proporcionado por el profesor:

```c
#include <ncurses.h>
#include <stdio.h>

int main() {
    initscr(); // Reserva memoria para la ventana principal
    printw("Hello World!"); // Lo escribe en la memoria
    refresh(); // Pasa lo que hay en memoria a la pantalla para que se vea
    getch();
    endwin(); // Cierra el modo ncurses
    return 0;
}
```

## 4.2. Ejemplo completo para *ej2*

```c
#include <stdlib.h>
#include <ncurses.h>

int main(void) {
    int rows, cols;

```

```
    initscr();

    if (has_colors() == FALSE) { // Comprobar soporte de color
        endwin();
        printf("Your␣terminal␣does␣not␣support␣color\n");
        exit(1);
    }

    start_color(); // Activar soporte color

    // Crear combinaci n de colores
    init_pair(1, COLOR_YELLOW, COLOR_GREEN); // Texto amarillo, fondo verde
    init_pair(2, COLOR_BLACK, COLOR_WHITE);
    init_pair(3,COLOR_WHITE,COLOR_BLUE);
    clear();

    refresh();
    getmaxyx(stdscr, rows, cols);

    WINDOW *window = newwin(rows,cols,0,0); // Crea una ventana del tama o de la
        terminal
    wbkgd(window, COLOR_PAIR(3));
    box(window, '|', '-'); // Dibuja el marco de la ventana

    mvwprintw(window, 10, 10, "una␣cadena");
    wrefresh(window);

    getch();
    endwin();
    return 0;
}
```

### 4.3. Ejemplo modificado de *ej2*

```
#include <stdlib.h>
#include <ncurses.h>

int main(void) {
    int rows, cols;

    initscr();

    if (has_colors() == FALSE) {
        endwin();
        printf("Your␣terminal␣does␣not␣support␣color\n");
        exit(1);
    }

    start_color();
    init_pair(1, COLOR_YELLOW, COLOR_GREEN);
    init_pair(2, COLOR_BLACK, COLOR_WHITE);
    init_pair(3,COLOR_WHITE,COLOR_BLUE);
    clear();

    refresh();
    //getmaxyx(stdscr, rows, cols);
    rows=10;
    cols=40;

    WINDOW *window = newwin(rows,cols,5,10); // Modificaci n del tama o
    wbkgd(window, COLOR_PAIR(1)); // Cambio de colores: amarillo y verde
    box(window, '*', '*'); // Modificaci n de marco
```

```

    mvwprintw(window, 10, 10, "una␣cadena");
    wrefresh(window);

    getch();
    endwin();
    return 0;
}
```

## 4.4. Ejemplo completo para *ej3*

```
#include <ncurses.h>
#include <unistd.h>

#define DELAY 30000

int main(int argc, char *argv[]) {
 int x = 0, y = 0;
 int max_y = 20, max_x = 20;
 int next_x = 0;
 int direction = 1;

 initscr();
 noecho(); // Para no mostrar por pantalla las teclas que se pulsan
 curs_set(FALSE); // Ocultar el cursor

 while(1) {
    clear();
    mvprintw(y, x, "o");
    refresh();

    usleep(DELAY); // Pausa para que nos de tiempo a ver la pelota

    next_x = x + direction;

    if (next_x >= max_x || next_x < 0) {
        direction*= -1;
    } else {
        x+= direction;
    }
 }

 endwin();
}
```

## 4.5. Código completo para el juego Pong

```
#include <ncurses.h>
#include <unistd.h>
#include <stdlib.h>

#define DELAY 50000 // Velocidad de la pelota
#define LONGITUD_PALA 5 // Altura de las palas

int main(int argc, char *argv[]) {
 // Dimensiones del tablero
 int max_y=30, max_x=60;

 // Inicializar entorno de ncurses
 initscr();
 start_color(); // Activar soporte de color
```

```
cbreak();
noecho(); // Para no mostrar por pantalla las teclas que se pulsan
keypad(stdscr, TRUE);
curs_set(FALSE); // Ocultar el cursor
nodelay(stdscr, TRUE);

// Configurar los colores
init_pair(1, COLOR_WHITE, COLOR_BLACK);
init_pair(2, COLOR_GREEN, COLOR_BLACK);
init_pair(3, COLOR_YELLOW, COLOR_BLACK);
init_pair(4, COLOR_CYAN, COLOR_BLACK);

// Variables del juego
int pelota_x = max_x /2;
int pelota_y = max_y /2;
int dir_x = 1;
int dir_y = 1;

// Posicion de las palas
int pala_izq_y = (max_y /2) - (LONGITUD_PALA /2);
int pala_der_y = (max_y /2) - (LONGITUD_PALA /2);

// Puntuacion
int puntos_j1 = 0;
int puntos_j2 = 0;

while(1) {
   // Leer entrada de teclado
   int tecla = getch();

   // Jugador 1
   if(tecla == 'w' || tecla == 'W'){
       if(pala_izq_y > 1){
           pala_izq_y--;
       }
   }
   if(tecla == 's' || tecla == 'S'){
       if(pala_izq_y < max_y - LONGITUD_PALA -1){
           pala_izq_y++;
       }
   }

   // Jugador 2
   if(tecla == KEY_UP){
       if(pala_der_y > 1){
           pala_der_y--;
       }
   }
   if(tecla == KEY_DOWN){
       if(pala_der_y < max_y - LONGITUD_PALA -1){
           pala_der_y++;
       }
   }

   // Salir del juego
   if(tecla == 'q' || tecla == 'Q'){
       break;
   }

   // Actualizar posicion de la pelota
   pelota_x += dir_x;
   pelota_y += dir_y;

```

```
    // Colisiones
    // Rebote en techo y suelo
    if(pelota_y <=1 || pelota_y >= max_y -2){
        dir_y *= -1;
    }

    // Rebote en la pala izquierda
    if (pelota_x == 3 && pelota_y >= pala_izq_y && pelota_y < pala_izq_y +
        LONGITUD_PALA) {
        dir_x *= -1;
    }

    // Rebote en la pala derecha
    else if (pelota_x == max_x - 4 && pelota_y >= pala_der_y && pelota_y < pala_der_y
         + LONGITUD_PALA) {
        dir_x *= -1;
    }

    // Puntuacion
    if (pelota_x <= 0) { // Punto para el jugador 2
            puntos_j2++;
            pelota_x = max_x / 2; pelota_y = max_y / 2;
            dir_x = 1;
    } else if (pelota_x >= max_x - 1) { // Punto para el jugador 1
            puntos_j1++;
            pelota_x = max_x / 2; pelota_y = max_y / 2;
            dir_x = -1;
    }

    // Dibujar los elementos en pantalla
        clear();

        // Marco
        attron(COLOR_PAIR(4));
        for(int i = 0; i < max_x; i++) {
            mvprintw(0, i, "-");          // Techo
            mvprintw(max_y - 1, i, "-");  // Suelo
        }
        for(int i = 0; i < max_y; i++) { // Paredes
            mvprintw(i, 0, "|");          // Pared izquierda
            mvprintw(i, max_x - 1, "|");  // Pared derecha
        }
        for(int i = 1; i < max_y - 1; i++) {// La red del centro
            mvprintw(i, max_x/2, "|");
        }
        attroff(COLOR_PAIR(4));

        // Marcador
        attron(COLOR_PAIR(3));
        mvprintw(1, max_x/2 - 15, "␣Jugador␣1:␣%d␣", puntos_j1);
        mvprintw(1, max_x/2 + 2, "␣Jugador␣2:␣%d␣", puntos_j2);
        attroff(COLOR_PAIR(3));

        // Palas
        attron(COLOR_PAIR(2));
        for(int i = 0; i < LONGITUD_PALA; i++) {
            mvprintw(pala_izq_y + i, 2, "#");          // Pala izquierda
            mvprintw(pala_der_y + i, max_x - 3, "#"); // Pala derecha
        }
        attroff(COLOR_PAIR(2));

        // Pelota
        attron(COLOR_PAIR(1));
```

```
        mvprintw(pelota_y, pelota_x, "o");
        attroff(COLOR_PAIR(1));
    refresh();

    usleep(DELAY); // Pausa para que nos de tiempo a ver la pelota

 }

 // Finalizar
 endwin();
 return 0;
}
```

## 4.6. Código completo para Pong con pantallas

```
#include <ncurses.h>
#include <unistd.h>
#include <stdlib.h>

#define DELAY 45000          // Velocidad de la pelota
#define LONGITUD_PALA 5      // Altura de las palas
#define PUNTOS_PARA_GANAR 5  // Puntos necesarios para ganar la partida

// Funcion de pantalla de bienvenida
void pantalla_bienvenida(int max_y, int max_x){
    nodelay(stdscr, FALSE); // Para que getch() espere la pulsacion de una tecla
    clear();

    // Dibujar marco de presentacion
    attron(COLOR_PAIR(4));
    for(int i = 0; i < max_x; i++) { // Techo y suelo
        mvprintw(0, i, "-"); mvprintw(max_y - 1, i, "-");
    }
    for(int i = 0; i < max_y; i++) { // Paredes
         mvprintw(i, 0, "|"); mvprintw(i, max_x - 1, "|");
    }
    attroff(COLOR_PAIR(4));

    attron(COLOR_PAIR(3));
    mvprintw(max_y/2 - 5, max_x/2 - 6, "JUEGO␣PONG");
    attroff(COLOR_PAIR(3));

    mvprintw(max_y/2 - 3, max_x/2 - 15, "Autora:␣Ines␣Ruiz␣Sanchez");
    mvprintw(max_y/2 - 1, max_x/2 - 20, "J1␣(Pala␣Izquierda):␣W␣(Arriba)␣/␣S␣(Abajo)"
        );
    mvprintw(max_y/2 + 1, max_x/2 - 20, "J2␣(Pala␣Derecha):␣Flecha␣Arriba␣/␣Abajo");
    mvprintw(max_y/2 + 4, max_x/2 - 18, "Pulsa␣cualquier␣tecla␣para␣empezar␣...");

    refresh();
    getch(); // Esperar tecla
}

// Funcion de pantalla de finalizacion
int pantalla_final(int max_y, int max_x, int ganador, int puntos_j1, int puntos_j2)
    nodelay(stdscr, FALSE); // sperar a que el usuario responda
    clear();

    // Marco de la pantalla final
    attron(COLOR_PAIR(4));
    for(int i = 0; i < max_x; i++) { mvprintw(0, i, "-"); mvprintw(max_y - 1, i, "-")
        ; } // Techo y suelo
```

```
    for(int i = 0; i < max_y; i++) { mvprintw(i, 0, "|"); mvprintw(i, max_x - 1, "|") ; } // Paredes
    attroff(COLOR_PAIR(4));

    attron(COLOR_PAIR(3));
    mvprintw(max_y/2 - 4, max_x/2 - 8, "Fin del juego");
    attroff(COLOR_PAIR(3));

    if (ganador == 1) {
        mvprintw(max_y/2 - 2, max_x/2 - 15, "Felicidades Jugador 1, has ganado!");
    } else if (ganador == 2) {
        mvprintw(max_y/2 - 2, max_x/2 - 15, "Felicidades Jugador 2, has ganado!");
    } else {
        mvprintw(max_y/2 - 2, max_x/2 - 10, "Partida cancelada");
    }

    mvprintw(max_y/2, max_x/2 - 15, "Marcador Final");
    mvprintw(max_y/2, max_x/2 - 16, "J1: %d puntos | J2: %d puntos", puntos_j1, puntos_j2);
    mvprintw(max_y/2 + 3, max_x/2 - 17, "Pulsa S para volver a jugar y N para terminar. ");
    refresh();

    // Captar respuesta
    int respuesta;
    while(1) {
        respuesta = getch();
        if (respuesta == 'n' || respuesta == 'N' || respuesta == 'q' || respuesta == 'Q') {
            return 0;
        } else if (respuesta == 's' || respuesta == 'S') {
            return 1;
        }
    }
}

int main(int argc, char *argv[]) {
    // Dimensiones del tablero fijo
    int max_y = 30, max_x = 60;

    int jugar_de_nuevo = 1;

    // Inicializar entorno de ncurses
    initscr();
    start_color(); // Activar soporte de color
    cbreak();
    noecho(); // Para no mostrar por pantalla las teclas que se pulsan
    keypad(stdscr, TRUE); // Para detectar las flechas de direccion
    curs_set(FALSE); // Ocultar el cursor

    // Configurar los colores
    init_pair(1, COLOR_WHITE, COLOR_BLACK);
    init_pair(2, COLOR_GREEN, COLOR_BLACK);
    init_pair(3, COLOR_YELLOW, COLOR_BLACK);
    init_pair(4, COLOR_CYAN, COLOR_BLACK);

    // Bucle principal (Permite volver a jugar al terminar una partida)
    while (jugar_de_nuevo) {

        // Pantalla de bienvenida
        pantalla_bienvenida(max_y, max_x);

        // Variables del juego
```

```
        int pelota_x = max_x / 2;   int pelota_y = max_y / 2;
        int dir_x = 1;  int dir_y = 1;

        // Posicion de las palas
        int pala_izq_y = (max_y / 2) - (LONGITUD_PALA / 2); // Pala izq
        int pala_der_y = (max_y / 2) - (LONGITUD_PALA / 2); // Pala der

        // Puntuacion
        int puntos_j1 = 0;
        int puntos_j2 = 0;
        int ganador = 0; // 0 = Ninguno, 1 = J1, 2 = J2

        // Modo no bloqueante para el juego
        nodelay(stdscr, TRUE);

        // Logica del juego
        while (1) {
            // Leer entrada del teclado
            int tecla = getch();

            // Jugador 1 (W/S)
            if((tecla == 'w' || tecla == 'W') && (pala_izq_y > 1)){ pala_izq_y--; }
            if((tecla == 's' || tecla == 'S') && (pala_izq_y < max_y - LONGITUD_PALA - 1)){ pala_izq_y++; }

            // Jugador 2 (Flechas)
            if((tecla == KEY_UP) && (pala_der_y > 1)){ pala_der_y--; }
            if((tecla == KEY_DOWN) && (pala_der_y < max_y - LONGITUD_PALA - 1)){ pala_der_y++; }

            // Salir del juego
            if(tecla == 'q' || tecla == 'Q'){ break; }

            // Actualizar posicion de la pelota
            pelota_x += dir_x; pelota_y += dir_y;

            // Colisiones
            // Rebote en techo y suelo
            if(pelota_y <= 1 || pelota_y >= max_y - 2) dir_y *= -1;

            // Rebote en pala izquierda
            if (pelota_x == 3 && pelota_y >= pala_izq_y && pelota_y < pala_izq_y + LONGITUD_PALA) dir_x *= -1;

            // Rebote en pala derecha
            else if (pelota_x == max_x - 4 && pelota_y >= pala_der_y && pelota_y < pala_der_y + LONGITUD_PALA) {
                dir_x *= -1;
            }

            // Puntuacion
            if (pelota_x <= 0) { // Punto para el ugador 2
                puntos_j2++;
                pelota_x = max_x / 2; pelota_y = max_y / 2;
                dir_x = 1;
                if (puntos_j2 >= PUNTOS_PARA_GANAR) { ganador = 2; break; }
            } else if (pelota_x >= max_x - 1) { // Punto ppara jugador 1
                puntos_j1++;
                pelota_x = max_x / 2; pelota_y = max_y / 2;
                dir_x = -1;
                if (puntos_j1 >= PUNTOS_PARA_GANAR) { ganador = 1; break; }
            }

```

```
            // Dibujar
            clear();

            // Marco
            attron(COLOR_PAIR(4));
            for(int i = 0; i < max_x; i++) { mvprintw(0, i, "-"); mvprintw(max_y - 1, i, "-"); } // Techo
            for(int i = 0; i < max_y; i++) { mvprintw(i, 0, "|"); mvprintw(i, max_x - 1, "|"); } // Paredes
            for(int i = 1; i < max_y - 1; i++) { mvprintw(i, max_x/2, "|"); } // Red del centro
            attroff(COLOR_PAIR(4));

            // Marcador
            attron(COLOR_PAIR(3));
            mvprintw(1, max_x/2 - 15, "␣Jugador␣1:␣%d␣", puntos_j1);
            mvprintw(1, max_x/2 +  2, "␣Jugador␣2:␣%d␣", puntos_j2);
            attroff(COLOR_PAIR(3));

            // Palas
            attron(COLOR_PAIR(2));
            for(int i = 0; i < LONGITUD_PALA; i++) {
                mvprintw(pala_izq_y + i, 2, "#");
                mvprintw(pala_der_y + i, max_x - 3, "#");
            }
            attroff(COLOR_PAIR(2));

            // Pelota
            attron(COLOR_PAIR(1));
            mvprintw(pelota_y, pelota_x, "o");
            attroff(COLOR_PAIR(1));

            refresh();
            usleep(DELAY);
        }

        // Pantalla final
        jugar_de_nuevo = pantalla_final(max_y, max_x, ganador, puntos_j1, puntos_j2);
    }

    // Finalizar
    endwin();
    return 0;
}
```

Para compilar hay que usar la siguiente orden[17]:

```
gcc juego.c -o juego -lncurses
```

Listing 17: Compilación juego.c